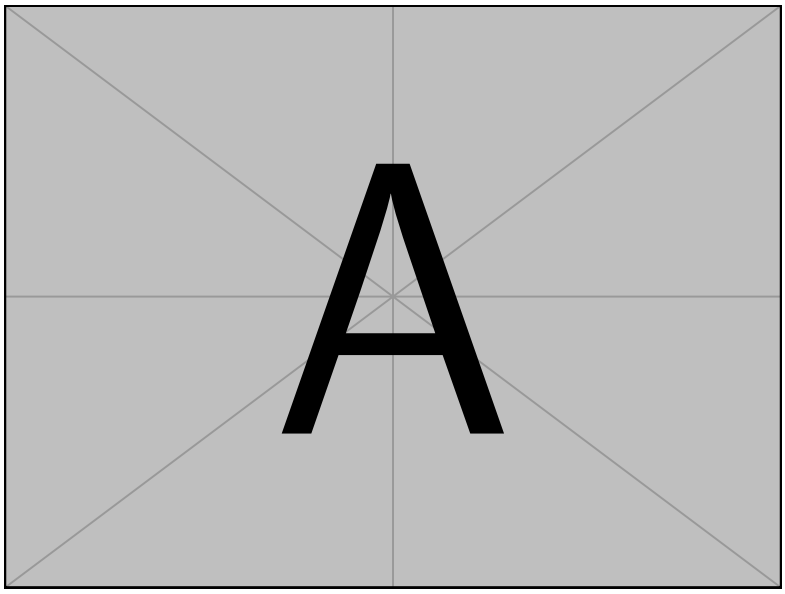
TECHNISCHE UNIVERSITÄT DRESDEN



Schaltkreis- und Systementwurf

Praktikumsbericht

Justus Gottfried Hache (Matrikelnummer)

Studiengang Informationssystemtechnik
Fakultät Elektrotechnik und Informationstechnik
Technische Universität Dresden

28. Oktober 2026

Inhaltsverzeichnis

1. Einleitung **4**

2. Beschreibung des Algorithmus **5**
2.1. Arithmetik 5

3. Entwurf **6**
3.1. Ablaufdiagramm 6
3.2. Unoptimierter Datenflussgraph 6
3.3. Unoptimierte FSM 6
3.4. Optimierungen 7
3.5. Unoptimierter komplexer Datenflussgraph 8
3.6. Unoptimierte komplexe FSM 8
3.7. Optimierter Datenflussgraph 8
3.8. Datenpfadarchitektur 8
3.9. Registertransferfolgen 11
3.10. Zustandsgraph 15
3.11. Gesamtschaltung 15

4. Simulation **16**
4.1. Verifikation 16
4.2. Schaltungssynthese 16
4.3. Layoutsynthese 16

5. Zusammenfassung **17**

A. Quellcode **18**
A.1. Referenzimplementierung 18
A.2. FSM 18
A.3. Controllogic 18

B. Reports **19**
B.1. Area 19
B.2. Power 19
B.3. Timing 19

1. Einleitung

Diese Arbeit dokumentiert das Design eines double-percision FFT-ASICs er wurde für das Modul „Schaltkreis und Systementwurf“ entwickelt. Die Aufgabenstellung umfasst dabei die Analyse des gewählten Algorithmus, eine sequentielle Referenzimplementierung, planung der Implementierung in Hardware mithilfe von Registertransferfolgen, sowie die tatsächliche Implementierung mit dem Cadence Design Framework 2. Im finalen schritt wird das Design syntetisiert, simuliert und verifiziert.

Die Fourier Transformation ist eine der wichtigsten Funktionen in der heutigen Zeit. Sie wird für Komprimierungsalgorithmen, Audio und Bildbearbeitung, Analyse von Signalen, Übertragung von Signalen und noch vielen anderen Bereichen verwendet. Die zentrale Eigenschaft der Fourier Transformation ist dabei, aus einer zeitlichen Folge die Amplituden der Frequenzkomponenten zu berechnen. Lange Zeit war es nur möglich, die diskrete Version der Fourier Transformation mit einer Laufzeit von $O = N^2$ zu berechnen. Erst in den 60er Jahren änderte sich dies mit einem der Schnellen Fourier Transformation, beschrieben in Die Zentrale Erkenntnis dabei war, dass man sich durch die periodische Natur der Exponentialfunktion viele Rechnungen sparen konnte. In ihrer Arbeit beschreiben Colley und Turkey, wie durch umformen der gleichungen die Fourier Transformation so geschrieben werden kann, dass nun eine Laufzeit von $O = N \log_N$erreicht werden kann. Dies ermöglicht es auch lange Folgen mit hoher Frequenzauflösung zu berechnen und heutzutage z.B. die Verwendung in Komprimierungsalgorithmen und die Fähigkeit in Echtzeit auf eingebetteten systemen zu laufen.Diese Arbeit wird sich damit befassen, wie man einen AISC (Application Specific Integrated Circuit) erstellen kann, welcher diese Berechnung noch schneller durchführen kann, als es ein Prozessor in gleicher Technologie könnte.

Zitiert wird so: [**munafo_escape_2023**]. Die `.bib` Datei wurde mit Zotero erzeugt. Bitte beim Export auf das neue Format bib**la**tex achten.

2. Beschreibung des Algorithmus

Wie in der Einleitung beschrieben, berechnet die FFT effizient aus einer Zeitlichen Folge an Werten die Frequenzkomponenten. Die finale Implementierung ist außerdem darauf ausgelegt, dass die Berechnung in-place stattfindet, also kein Zusätzlicher Speicher als der Eingabespeicher, welcher gleichzeitig Ausgabespeicher ist, benötigt wird.

Eingabe Der Algorithmus benötigt als Eingabe eine Liste an Werten, welche in gleichmäßigen Abständen aufgenommen wurden und als weitere Eingabe muss die länge der Liste angegeben werden. Die Länge der Liste kann dabei nur eine Potenz von 2 sein. Sollte die Länge der Zahlenfolge kürzer sein, so muss der restliche Speicher mit Nullen aufgefüllt werden.

Ausgabe Der Algorithmus liefert eine Liste an Werten, welche den Amplituden der Frequenzanteile entspricht. Diese Liste hat dabei die gleiche Länge, wie die Eingabe.

Implementierung Ein einfacher weg, die FFT zu veranschaulichen ist der rekursive ansatz. Dieser funktioniert, indem die Originale Zahlenfolge in 2 Teile zerlegt wird, ein Teil mit den Graden Indizes und ein Teil mit den ungeraden. Für jeden dieser Teile wird nun selbst eine FFT berechnet, die Ergebnisse werden schließlich miteinander verrechnet. Wenn die beiden vorherigen Ergebnisse $E[k]$ und $O[k]$ sind, gilt dann für das Gesamtergebnis X mit der Länge N:

$$\begin{aligned} X[k] &= E[k] + W_N^k O[k] \\ X[k + N/2] &= E[k] - W_N^k O[k] \\ k &= 0, \ldots, N/2 - 1 \end{aligned} \tag{1}$$

Man benötigt also für die Kombination N komplexe Multiplikationen und Additionen. Durch wiederholtes Teilen zerlegt man die Zahlenfolge nun solange bis nur noch ein Element übrig ist. Dies wird erreicht, nach $\log_2 N$ Teilen. Interessanterweise ist die Teilung in 2 Teile nicht der Effizienteste Weg, wenn man nur auf die Komplexen Multiplikationen schaut. Laut dem originalem Paper TODO: reference ist eine Teilung in 3 Teile 6% effizienter, eine Teilung in 4 gleich schnell im Vergleich zu einer Teilung in 2. Allerdings Bringt die Teilung in 2 deutlich vereinfachungen der Speicherung und Adressberechnung auf binärkomputern, sodass heutzutage meist eine Teilung in 2, einige praktische Implementierungen TODO:reference aber auch eine Teilung in 4 für eine bessere Hardwarenutzung verwenden. Die Referenzimplementierung TODO: link verwendet eine sequentielle Version dieses Algorithmus. Dabei wird jedes Mal Teilen einer neuen „Stufe“ zugeordnet. Die erste Stufe kombiniert dann N Teile, zu $N/2$ die letzte Stufe kombiniert lediglich 2. Dadurch kann eine feste Schleife erstellt werden, welche den Anfangsindex, der jeweiligen in einer Stufe berechnet und in einer zweiten schleife, wird dann durch die Elemente eines Teils gegangen.

2.1. Arithmetik

Was zum Rundungsverhalten/Abweichungen erklären, usw.

3. Entwurf

3.1. Ablaufdiagramm

In Diagramm **??** ist der Programmablauf graphisch dargestellt. Für eine performante Implementierung in Hardware gestaltet sich vor allem die 3 ineinanderliegenden Schleifen, welche in der Finalen implementierung teilweise parallel ausgeführt werden müssen. Im Ablaufdiagramm wurden folgende Variablen verwendet:

- len = der Logarithmus zur Basis 2 der konfigurierbaren Länge der Datenserie,
- N = die Länge der Datenserie am Eingang,
- $stagepow$ = 2 hoch die aktuelle Stufe. Wird als counter für den äußeren loop verwendet.
- $half$ = Hälfte von $stagepow$, oder der Wert aus der vorherigen Schleifeniteration.
- k = Zähler der mittleren Schleife.
- j = Zähler der inneren Schleife.
- $omega$ = Komplexe Zahl, entspricht W_N^k aus Kapitel Implementierung
- $domega$ = Komplexe Zahl, entspricht W_N^1 aus Kapitel Implementierung.
- t,u = Komplexe Hilfsvariablen

3.2. Unoptimierter Datenflussgraph

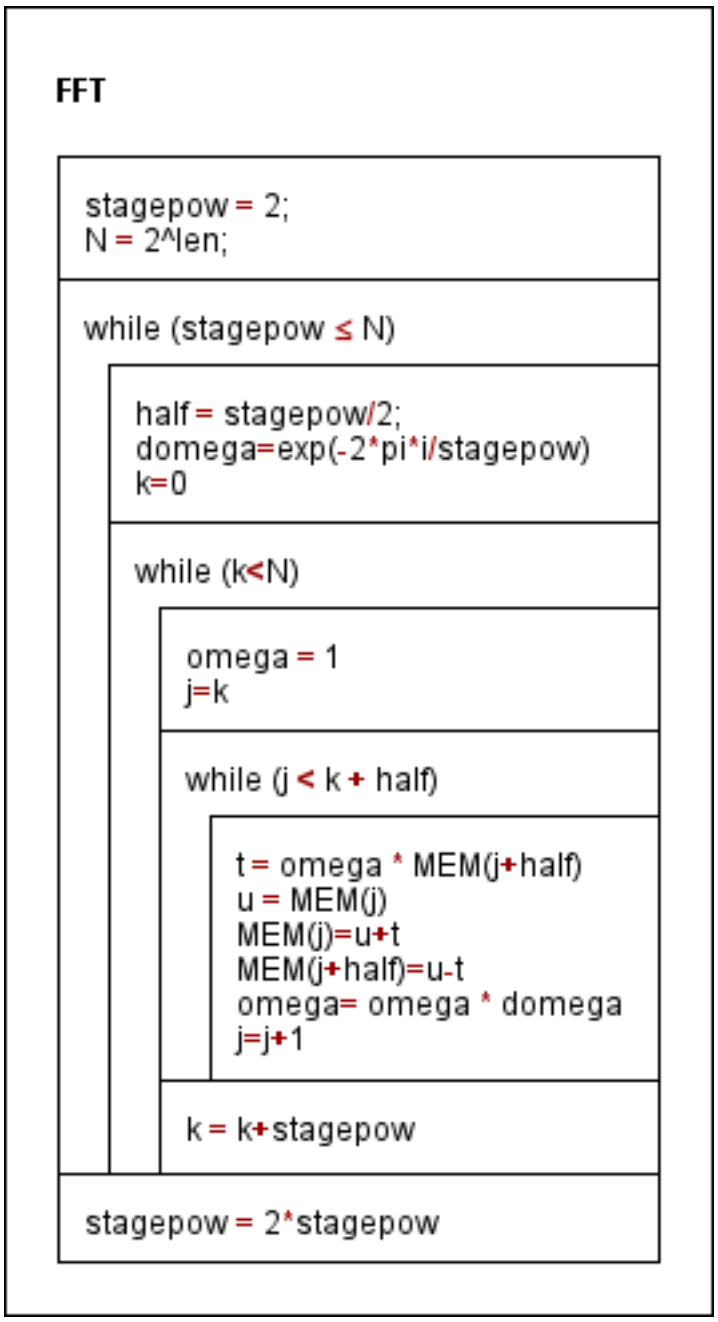
In Diagramm **??** is eine erste Datenpfadarchitektur dargestellt, welche als Startpunkt des Entwurfes dient. Prinzipiell kann man mit dieser Architektur alle Operationen ausführen, die für die Schnelle Fouriertransformation nötig wären. Eine solche Ausführung wäre allerdings nicht sonderlich schneller, als eine Implementierung auf einem üblichen Prozessor.

3.3. Unoptimierte FSM

Um die in Diagramm **??** dargestellte Datenpfadarchitektur zum berechnen einer Fourier zu verwenden, müssen die verschiedenen möglichen Datentransfers mit einem Endlichen Zustandsautomat (engl. Finite State Machine, FSM) gesteuert werden. Dieser wird im Folgenden mithilfe von Registertransferfolgen ersetllt. Dieser unoptimierte Zustandsautomat ist jedoch sehr groß, sodass wir die Funktionen komplexe Multiplikation, Addition sowie Subtraktion abgetrennt betrachten.

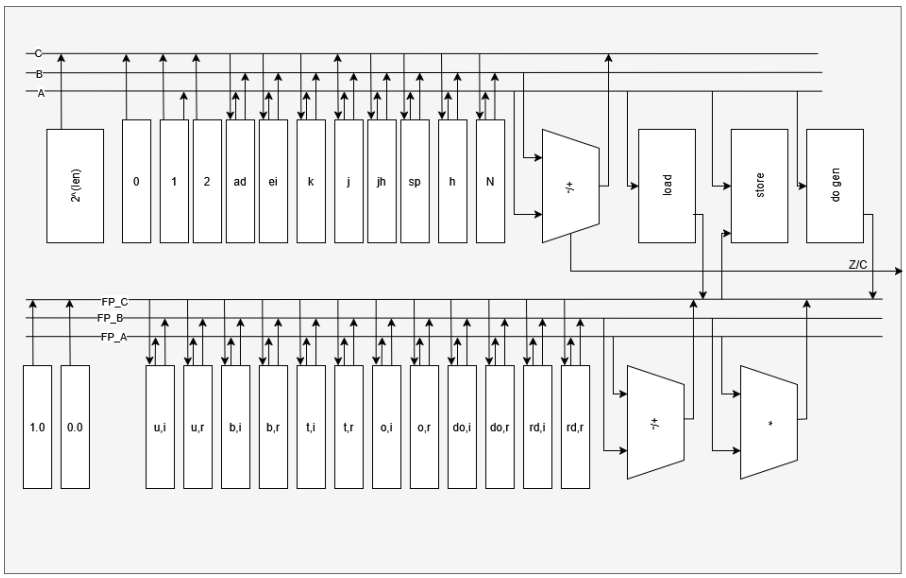
In der Registertransferfolge **??** sind jeweils die Zustände, welche für Komplexe Addition, Subtraktion und Multiplikation notwendig sind dargestellt. Die Parameter a,b und c stehen dafür für zwei Eingangs- und ein Ausgangsregisterpaar mit je einem FP64 Register für komplexen und reellen Teil. Schon hier wird deutlich, wie viele Takte eine einzelne komplexe Multiplikation einnehmen kann.

In der weiteren Registertransferfolge **??** ist der Zustandsautomat und die benötigten Transfere für die unoptimierte Datenpfadarchitektur dargestellt. Die im vorherigen Kapitel definierten Funktionen werden dabei aus Übersichtsgründen nur aufgerufen, in einer implementierten FSM müssten sie aber direkt in die eigentliche FSM eingefügt werden.



3.4. Optimierungen

Das primäre Optimierungskriterium in dieser Arbeit ist Geschwindigkeit, der ASIC muss dafür also eine iteration des inneren loops so schnell wie möglich ausführen. Durch die unoptimierte FSM ist deutlich geworden, wie viel Takte eine einzelne Iteration der inneren Schleife dauern würde. Aus Gründen der Übersichtlichkeit und Länge der Arbeit wird hier darauf verzichtet, eine Registertransferanalyse vom ersten Datenpfad darzustellen anstelle dessen führen wir zuerst Optimierungen ein. Um die Anzahl der Zustände zu verringern und die Verarbeitungsgeschwindigkeit zu erhöhen wechseln wir auf einen Datenpfad, der direkt mit komplexen Zahlen arbeiten kann. Dafür halten alle Register, welche eine komplexe Zahl speichern 128 Bit, je 64 für Imaginär- und Realteil. Daraus folgen drei Konsequenzen: Zum einen muss das Speicherinterface 128 Bit Daten lesen und schreiben können, wobei wir einen dualport Speicher, welcher gleichzeitig schreiben und lesen kann verwenden. Es müssen aber auch Komplexe Addition/Subtraktion und Multiplikation mit komplexen Zah-



len arbeiten können. Um dies zu erreichen erstellen wir 2 neue Datenpfadelemente, dargestellt in Diagramm 1a und 1b. Die komplexe ALU kann wie ein normales Datenpfadelement verwendet werden und benötigt nur einen Takt, die komplexe Multiplikation hingegen benötigt 2 Takte, bis ein valides Ergebnis am Ausgang erscheint. Durch die Register am Ende der internen Gleitkomma-Multiplizierer, kann man aber jeden Takt eine neue Berechnung starten. Der erstellte komplexe Gleitkomma Multiplizierer hat einen großen Flächenverbrauch, da er 4 Gleitkomma Multiplizierer enthält, welche schon im unoptimierten Datenpfad die Einheiten mit dem größten Flächenverbrauch waren. Das primäre Optimierungskriterium dieser Arbeit ist jedoch Geschwindigkeit und wie später noch gezeigt wird, erreicht der ASIC final eine hundertprozentige Auslastung des komplexen Multiplizierers in der mittleren und inneren Schleife.

3.5. Unoptimierter komplexer Datenflussgraph

In Diagramm **??** ist der modifizierte Datenpfad dargestellt, welcher jetzt die komplexen Operationen nutzt.

3.6. Unoptimierte komplexe FSM

Mithilfe des komplexen Datenpfades und einiger kleinerer optimierungen in den Schleifen, erhält man das Registerfolgediagram **??**

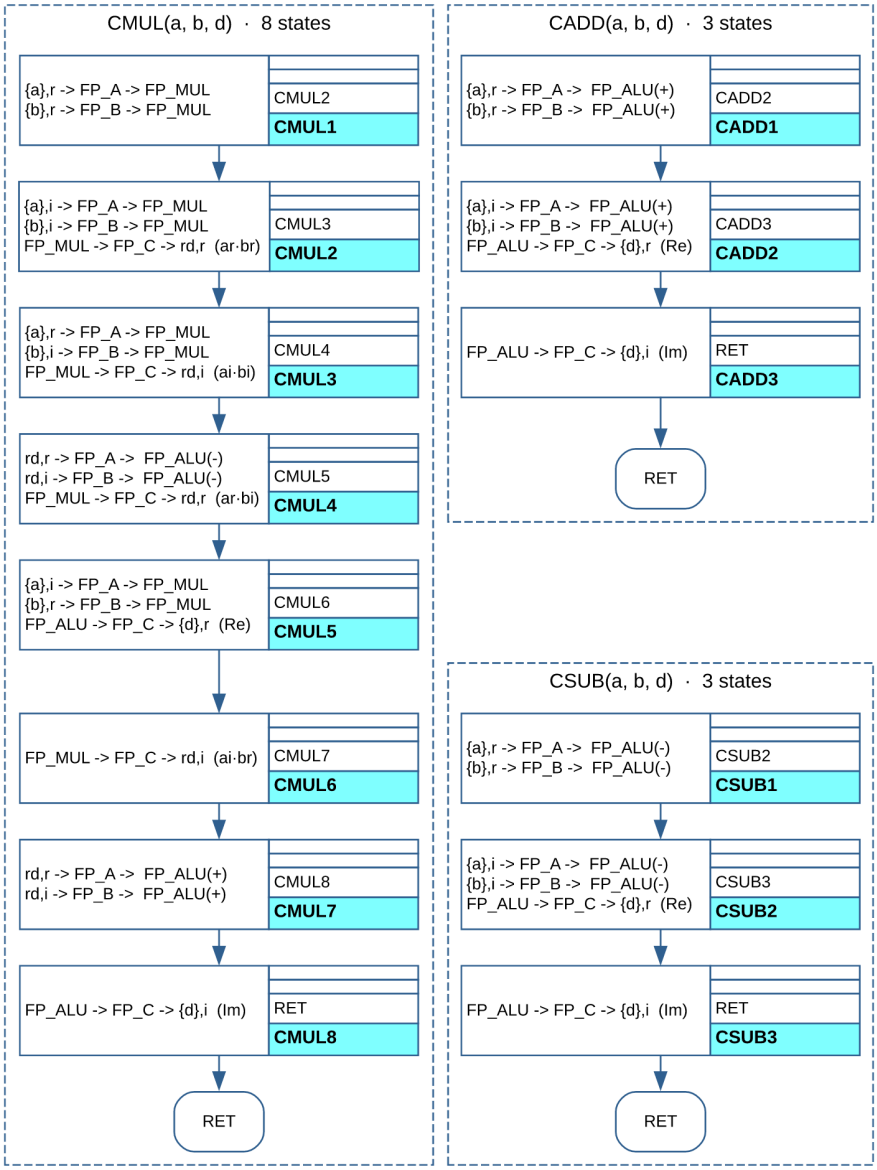
3.7. Optimierter Datenflussgraph

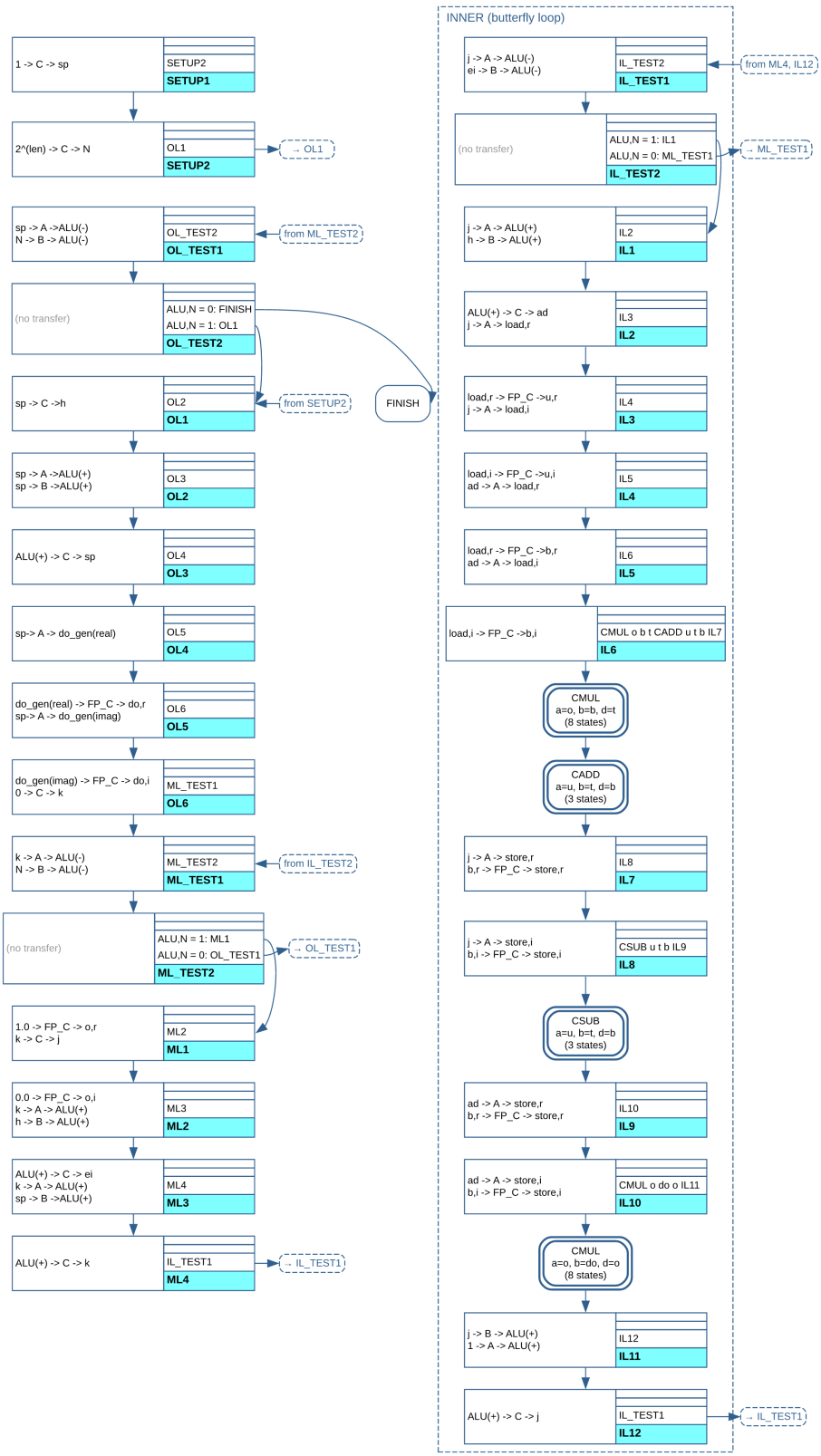
Auch erklären, was das Optimierungskriterium ist.

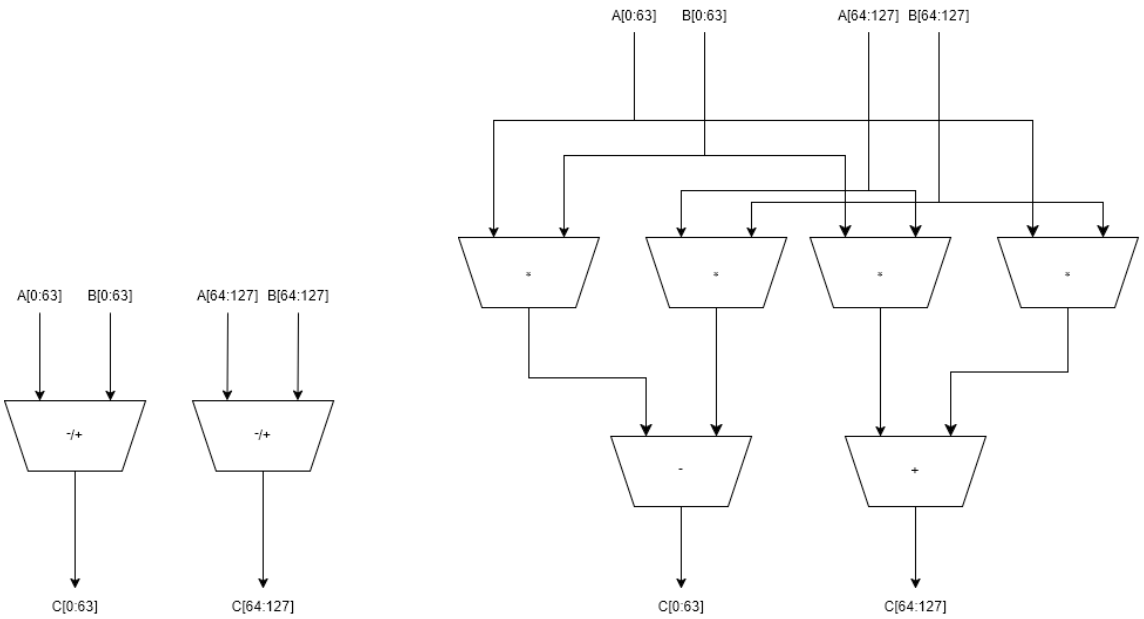
3.8. Datenpfadarchitektur

In Tabelle 1 sind die Datentransferoperationen für die Register, Recheneinheiten und den Speicher aufgelistet. Sie wurden aus dem Datenflussgraphen abgeleitet. Zeigt mehr als eine Quelle auf ein Datenpfadelement, wird ein Multiplexer benötigt.

Anschließend wurde die Datenpfadarchitektur erstellt. Als Bild genügt hier der Ausdruck aus Cadence.


CMUL(a, b, d) · 8 states
{a},r -> FP_A -> FP_MUL
{b},r -> FP_B -> FP_MUL
CMUL2
CMUL1
{a},i -> FP_A -> FP_MUL
{b},i -> FP_B -> FP_MUL
FP_MUL -> FP_C -> rd,r (ar·br)
CMUL3
CMUL2
{a},r -> FP_A -> FP_MUL
{b},i -> FP_B -> FP_MUL
FP_MUL -> FP_C -> rd,i (ai·bi)
CMUL4
CMUL3
rd,r -> FP_A -> FP_ALU(-)
rd,i -> FP_B -> FP_ALU(-)
FP_MUL -> FP_C -> rd,r (ar·bi)
CMUL5
CMUL4
{a},i -> FP_A -> FP_MUL
{b},r -> FP_B -> FP_MUL
FP_ALU -> FP_C -> {d},r (Re)
CMUL6
CMUL5
FP_MUL -> FP_C -> rd,i (ai·br)
CMUL7
CMUL6
rd,r -> FP_A -> FP_ALU(+)
rd,i -> FP_B -> FP_ALU(+)
CMUL8
CMUL7
FP_ALU -> FP_C -> {d},i (Im)
RET
CMUL8
RET
CADD(a, b, d) · 3 states
{a},r -> FP_A -> FP_ALU(+)
{b},r -> FP_B -> FP_ALU(+)
CADD2
CADD1
{a},i -> FP_A -> FP_ALU(+)
{b},i -> FP_B -> FP_ALU(+)
FP_ALU -> FP_C -> {d},r (Re)
CADD3
CADD2
FP_ALU -> FP_C -> {d},i (Im)
RET
CADD3
RET
CSUB(a, b, d) · 3 states
{a},r -> FP_A -> FP_ALU(-)
{b},r -> FP_B -> FP_ALU(-)
CSUB2
CSUB1
{a},i -> FP_A -> FP_ALU(-)
{b},i -> FP_B -> FP_ALU(-)
FP_ALU -> FP_C -> {d},r (Re)
CSUB3
CSUB2
FP_ALU -> FP_C -> {d},i (Im)
RET
CSUB3
RET


INNER (butterfly loop)
1 -> C -> sp
SETUP2
SETUP1
2^(len) -> C -> N
OL1
SETUP2
→ OL1
sp -> A ->ALU(-)
N -> B -> ALU(-)
OL_TEST2
OL_TEST1
from ML_TEST2
(no transfer)
ALU,N = 0: FINISH
ALU,N = 1: OL1
OL_TEST2
sp -> C ->h
OL2
OL1
from SETUP2
FINISH
sp -> A ->ALU(+)
sp -> B ->ALU(+)
OL3
OL2
ALU(+) -> C -> sp
OL4
OL3
sp-> A -> do_gen(real)
OL5
OL4
do_gen(real) -> FP_C -> do,r
sp-> A -> do_gen(imag)
OL6
OL5
do_gen(imag) -> FP_C -> do,i
0 -> C -> k
ML_TEST1
OL6
k -> A -> ALU(-)
N -> B -> ALU(-)
ML_TEST2
ML_TEST1
from IL_TEST2
(no transfer)
ALU,N = 1: ML1
ALU,N = 0: OL_TEST1
ML_TEST2
→ OL_TEST1
1.0 -> FP_C -> o,r
k -> C -> j
ML2
ML1
0.0 -> FP_C -> o,i
k -> A -> ALU(+)
h -> B -> ALU(+)
ML3
ML2
ALU(+) -> C -> ei
k -> A -> ALU(+)
sp -> B ->ALU(+)
ML4
ML3
ALU(+) -> C -> k
IL_TEST1
ML4
→ IL_TEST1
j -> A -> ALU(-)
ei -> B -> ALU(-)
IL_TEST2
IL_TEST1
from ML4, IL12
(no transfer)
ALU,N = 1: IL1
ALU,N = 0: ML_TEST1
IL_TEST2
→ ML_TEST1
j -> A -> ALU(+)
h -> B -> ALU(+)
IL2
IL1
ALU(+) -> C -> ad
j -> A -> load,r
IL3
IL2
load,r -> FP_C ->u,r
j -> A -> load,i
IL4
IL3
load,i -> FP_C ->u,i
ad -> A -> load,r
IL5
IL4
load,r -> FP_C ->b,r
ad -> A -> load,i
IL6
IL5
load,i -> FP_C ->b,i
CMUL o b t CADD u t b IL7
IL6
CMUL
a=o, b=b, d=t
(8 states)
CADD
a=u, b=t, d=b
(3 states)
j -> A -> store,r
b,r -> FP_C -> store,r
IL8
IL7
j -> A -> store,i
b,i -> FP_C -> store,i
CSUB u t b IL9
IL8
CSUB
a=u, b=t, d=b
(3 states)
ad -> A -> store,r
b,r -> FP_C -> store,r
IL10
IL9
ad -> A -> store,i
b,i -> FP_C -> store,i
CMUL o do o IL11
IL10
CMUL
a=o, b=do, d=o
(8 states)
j -> B -> ALU(+)
1 -> A -> ALU(+)
IL12
IL11
ALU(+) -> C -> j
IL_TEST1
IL12
→ IL_TEST1

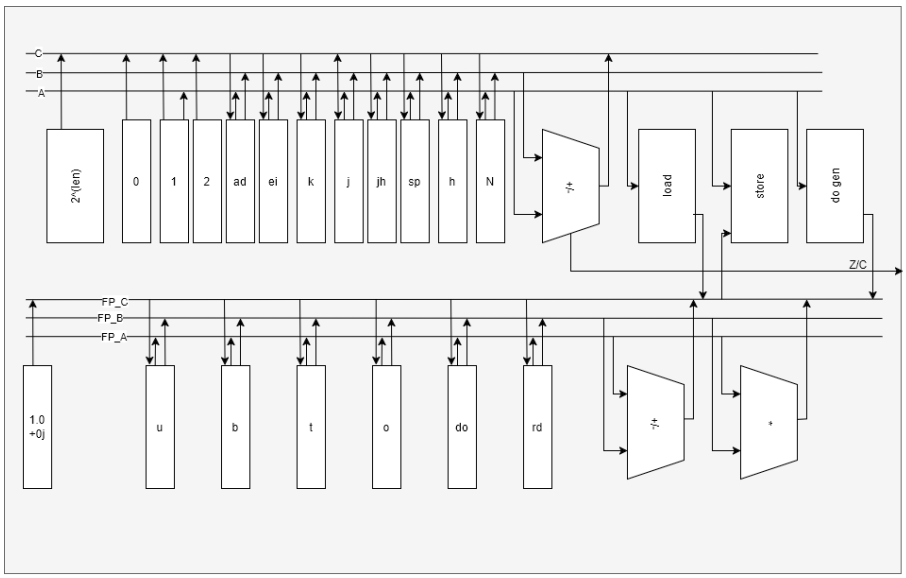


(a) Komplexe ALU (b) Komplexer Multiplizierer

Abbildung 1: Komplexe Datenpfadelemente

3.9. Registertransferfolgen

In Abbildung 2 seht ihr ein Beispiel der Registertransferfolgen, als formschöne Vektorgrafik. Erstellt in drawio. Für den Export wurden die Einstellungen in Abbildung 3 verwendet.



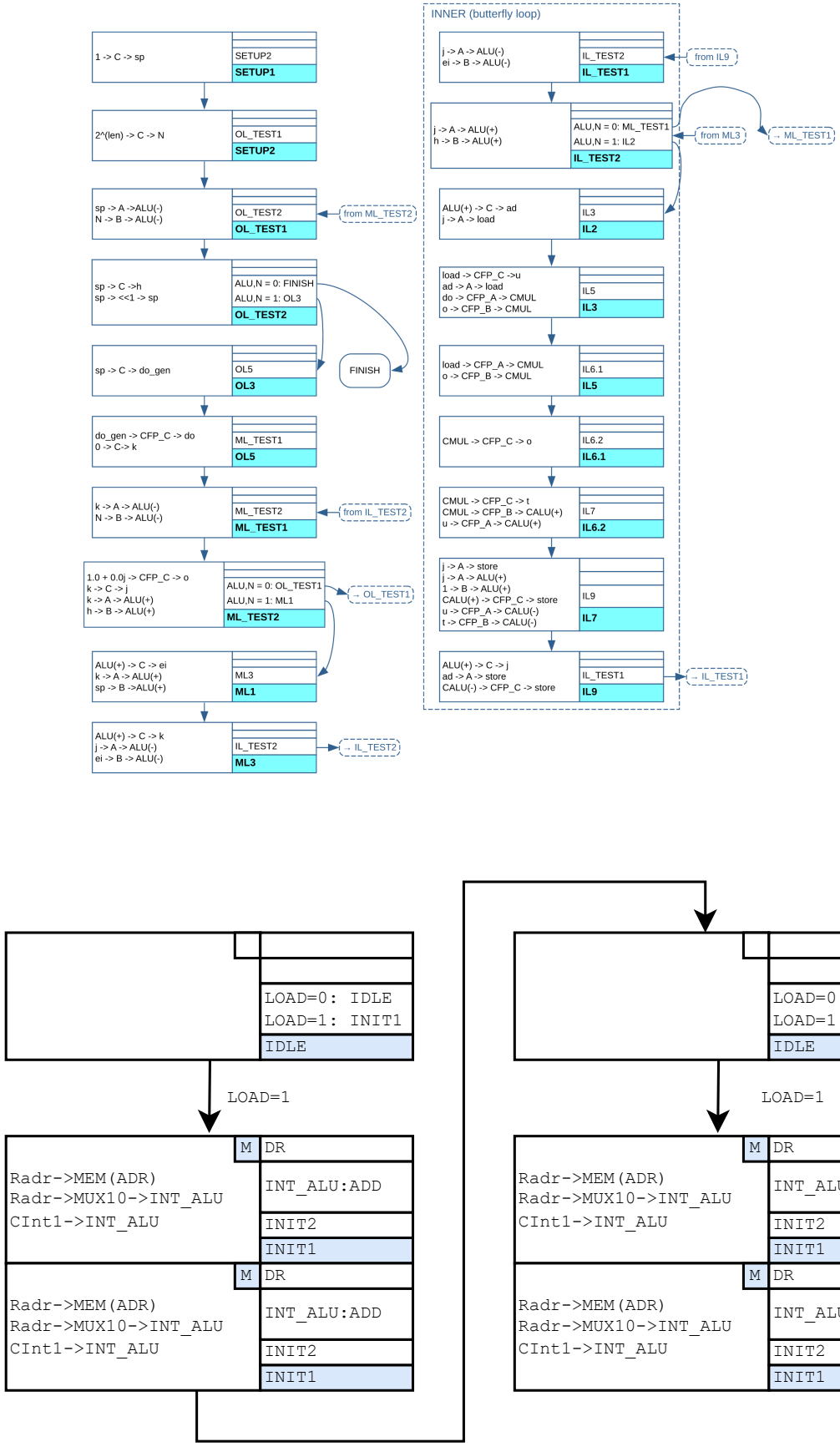


Abbildung 2: Registertransferfolgen. Erstellt mit drawio. Source befindet sich in dieser zip Datei.

Ziel	Quellen	1	2	3	4	5	6	7	8
R1	←	0	T1						
R3	←	0	T1						
RR	←	0	T1						
RC	←	0	T1						
R2	←	0	T2						
R4	←	0	T2						
RCX	←	T1							
RCY	←	T1							
RD	←	T3							
Rres	←	MEM(DO)							
Riter	←	MEM(DO)							
RI	←	Riter	T4						
Radr	←	T4							
MEM(ADR)	←	Radr							
MEM(DI)	←	RI							
ALU(1)	←	T1	T2	2	RC	R2	Rres	RR	
ALU(2)	←	T2	2	RCY	R4	RCX	1	4	T1
MUL(1)	←	2	RD	T1	T2				
MUL(2)	←	RR	RC	R3	R1	T1			
DIV(1)	←	4							
DIV(2)	←	MEM(DO)							
ALU_INT(1)	←	Radr	RI						
ALU_INT(2)	←	CInt 1							

Tabelle 1: Datentransferoperationen der Datenpfadarchitektur. Bei Konstanten wurde auf das Präfix *Const* verzichtet.

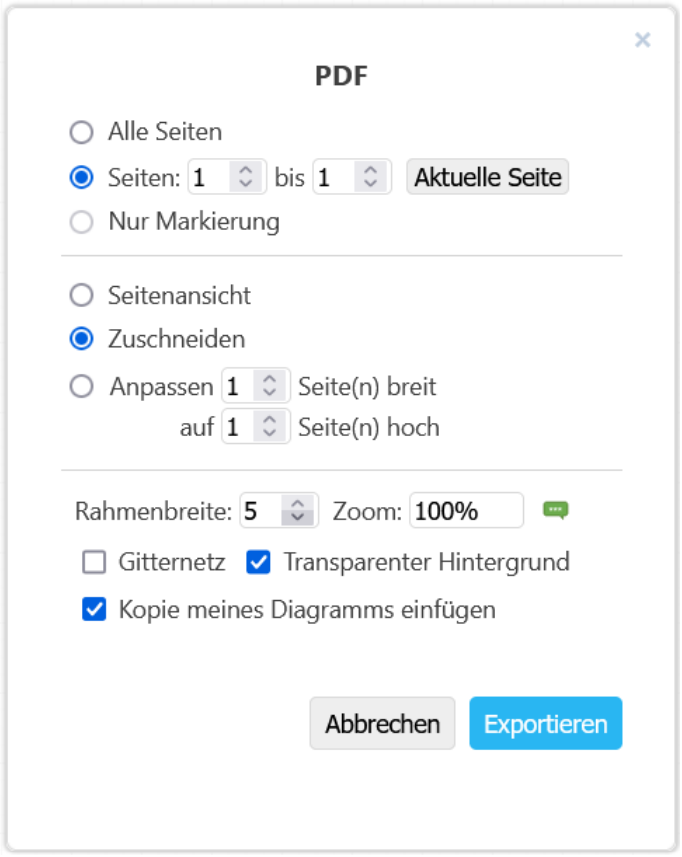


Abbildung 3: Exporteinstellungen drawio.

3.10. Zustandsgraph

Nicht vergessen: Die Zustandskodierung, wie in Tabelle 2 dargestellt.

Zustand	Hex-Code	Zustand	Hex-Code
IDLE	00	CLOOP2	0E
INIT1	01	CLOOP3	0F
INIT2	02	ILOOP1	10
INIT3	03	ILOOP2	11
INIT4	04	ILOOP3	12
INIT5	05	ILOOP4	13
INIT6	06	ILOOP5	14
INIT7	07	ILOOP6	15
INIT8	08	ILOOP7	16
INIT9	09	CLOOP4	17
RLOOP1	0A	CLOOP5	18
RLOOP2	0B	CLOOP6	19
RLOOP3	0C	RLOOP4	1A
CLOOP1	0D	RLOOP5	1B
		RLOOP6	1C

Tabelle 2: Zustandskodierung

3.11. Gesamtschaltung

4. Simulation

In den folgenden Unterkapiteln wurde die Gesamtschaltung mithilfe der Verilog Testbench simuliert. Dafür müssen die beiden Simulationsparameter *resolution* und *iteration count* im Speicher liegen. Diese werden in eine Textdatei geschrieben, welche anschließend von der Testbench eingelesen werden. Es wurden folgende Simulationsparameter verwendet:

Parameter	Dezimal	Hexadezimal	Adresse im Speicher
resolution	25	0x00000019	0x00000000
iteration count	50	0x00000032	0x00000001

4.1. Verifikation

Wie habt ihr Sichergestellt, dass euer Algorithmus tut, was er tut?

4.2. Schaltungssynthese

4.3. Layoutsynthese

5. Zusammenfassung

Die Simulation des Schaltkreises hat gezeigt, dass er wie erwartet funktioniert. Ihr solltet noch ein paar Worte sagen zu:

- Abweichungen zur Referenzimplementierung
- Trade-off Geschwindigkeit - Area
- Verbesserungsmöglichkeiten

Eine halbe Seite genügt.

Appendix

A. Quellcode

A.1. Referenzimplementierung

```
#include<stdio.h>

int main() {
  printf("Hello World\n");
  return 0;
}
```

A.2. FSM

A.3. Controllogic

B. Reports

B.1. Area

B.2. Power

B.3. Timing